```
# * LANGUAGE - PYTHON
# * GOOGLE COLAB -
# * I will share this notebook
# * To create new cell - Press Esc + B (After Esc)

# To comment -- Select and Then press (ctrl+/) together.

## We are using Notebook (most easisest way to see output), To run

## Usually, we write in python scripts (.py file) --> that we can r
## must have python installed..

### when you strat typing say dataframe, using up and down arrow ke


#### random, time, matplotlib


import random
import time
import matplotlib.pyplot as plt


## all of temp in degree celsius
min_temp = 40
max_temp = 110
threshold_temp = 90


## intialize data logs
timestamps = []
temperatures  = []
cooling_status = []


def add_two_number(a,b):
  ## function body
  sum_of_two_number = a+b
  return sum_of_two_number
```

```
result = add_two_number(2,10)
print(f"Sum of two number is {result}")
```

```
Sum of two number is 12
```

```
def simulate_temperature():
  return random.uniform(min_temp, max_temp)


## call the function
simulate_temperature()
```

```
90.0976093575375
```

```
for i in range(30):
  current_time = i
  temp = simulate_temperature()

  if temp>threshold_temp:
    cooling="ON"
  else:
    cooling="OFF"

  temperatures.append(temp)
  timestamps.append(current_time)
  cooling_status.append(cooling)


  print(f"Time {current_time}s | Temp: {temp:.2f}°C | Cooling: {co

  time.sleep(2)
```

```
Time 0s | Temp: 74.95°C | Cooling: OFF
Time 1s | Temp: 88.50°C | Cooling: OFF
Time 2s | Temp: 72.58°C | Cooling: OFF
Time 3s | Temp: 90.89°C | Cooling: ON
Time 4s | Temp: 104.31°C | Cooling: ON
Time 5s | Temp: 105.61°C | Cooling: ON
Time 6s | Temp: 65.41°C | Cooling: OFF
Time 7s | Temp: 93.93°C | Cooling: ON
Time 8s | Temp: 45.71°C | Cooling: OFF
```

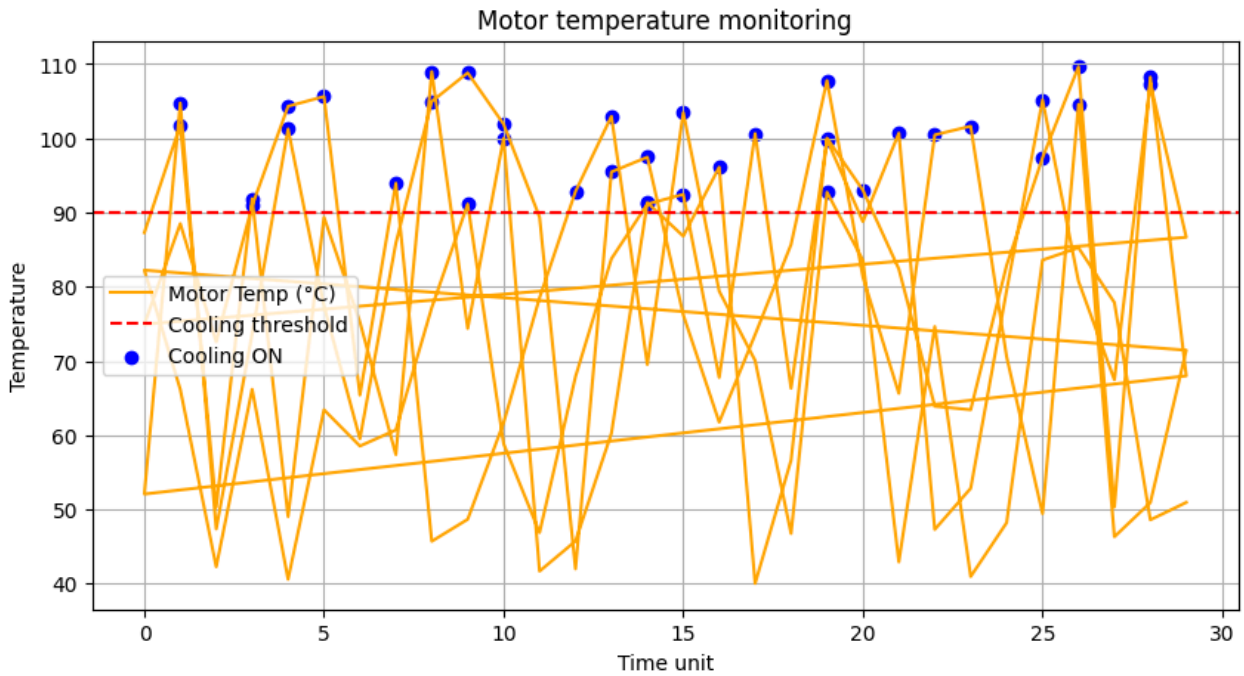
```
Time 9s | Temp: 48.67°C | Cooling: OFF
Time 10s | Temp: 61.68°C | Cooling: OFF
Time 11s | Temp: 78.51°C | Cooling: OFF
Time 12s | Temp: 92.82°C | Cooling: ON
Time 13s | Temp: 102.95°C | Cooling: ON
Time 14s | Temp: 69.52°C | Cooling: OFF
Time 15s | Temp: 103.46°C | Cooling: ON
Time 16s | Temp: 79.40°C | Cooling: OFF
Time 17s | Temp: 70.05°C | Cooling: OFF
Time 18s | Temp: 46.76°C | Cooling: OFF
Time 19s | Temp: 92.74°C | Cooling: ON
Time 20s | Temp: 83.56°C | Cooling: OFF
Time 21s | Temp: 42.93°C | Cooling: OFF
Time 22s | Temp: 74.64°C | Cooling: OFF
Time 23s | Temp: 40.96°C | Cooling: OFF
Time 24s | Temp: 48.21°C | Cooling: OFF
Time 25s | Temp: 83.59°C | Cooling: OFF
Time 26s | Temp: 85.26°C | Cooling: OFF
Time 27s | Temp: 77.87°C | Cooling: OFF
Time 28s | Temp: 48.60°C | Cooling: OFF
Time 29s | Temp: 50.95°C | Cooling: OFF
```

```
len(timestamps),len(cooling),len(temperatures)
```

```
(120, 3, 120)
```

```
plt.figure(figsize=(10,5))
plt.plot(timestamps, temperatures, label = "Motor Temp (°C)", colo
plt.axhline(y=threshold_temp, color='red',linestyle='--',label='Co

cooling_times = []
for t,c in zip(timestamps,cooling_status):
  if c=="ON":
    cooling_times.append(t)

cooling_temp = []
for temp,c in zip(temperatures,cooling_status):
  if c == "ON":
    cooling_temp.append(temp)
```

```
plt.scatter(cooling_times,cooling_temp,color='blue',label="Coolin
plt.xlabel('Time unit')

plt.ylabel('Temperature')

plt.title('Motor temperature monitoring')
plt.legend()
plt.grid(True)
plt.show()
```



Start coding or generate with AI.

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
time_steps = 100
production_data = {
    'Time':np.arange(time_steps),
    'Units Produced': np.random.randint(80,120,size=time_steps),
    'Defective Units': np.random.randint(0,10,size=time_steps)
}
```

```
type(production_data)
```

```
dict
```

```
production_data.keys()
```

```
dict_keys(['Time', 'Units Produced', 'Defective Units'])
```

```
from pprint import pprint
pprint(production_data)
```

```
{'Defective Units': array([0, 4, 2, 0, 3, 4, 6, 0, 2, 1, 8, 9
            1, 9, 0, 7, 0, 8, 5, 6, 9, 6, 9, 2, 1, 8, 7, 9, 6, 8,
            2, 6, 1, 1, 6, 5, 2, 8, 9, 5, 9, 9, 5, 0, 3, 9, 5, 5,
            4, 6, 3, 5, 3, 2, 6, 7, 3, 1, 9, 2, 0, 7, 2, 9, 6, 9,
            8, 4, 0, 9, 9, 0, 1, 5, 8, 7, 4, 0]),
 'Time': array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 1
            17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30
            34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47
            51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64
            68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81
            85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98
 'Units Produced': array([ 88,  85,  95, 108,  82,  99, 115,
             86, 112, 119, 118,  97, 119,  80,  90, 107, 104, 102,
            114,  86,  95, 105,  81,  80,  91,  84, 116, 111,  88,
             95,  82,  99, 103, 112, 103,  90,  87, 115, 117, 119,
            104, 114, 104, 108,  97,  97,  81, 114,  95, 115, 112,
             93, 100,  99,  87,  86,  82,  96, 112,  91, 101, 101,
            117,  87, 106, 106, 113, 100, 109, 112, 107, 112,  84,
            114,  96, 107, 109, 108,  85, 114, 116, 103])}
```

```
import pandas as pd
```

```
df = pd.DataFrame(production_data)
df
```

	Time	Units Produced	Defective Units
0	0	88	0
1	1	85	4
2	2	95	2
3	3	108	0
4	4	82	3
...	...	...	...
95	95	108	5
96	96	85	8
97	97	114	7
98	98	116	4
99	99	103	0

100 rows × 3 columns

```
df['Yield'] = df['Units Produced'] - df['Defective Units']
df['Efficency'] = (df['Yield']/df['Units Produced'])*100
```

```
df.head(5)
```

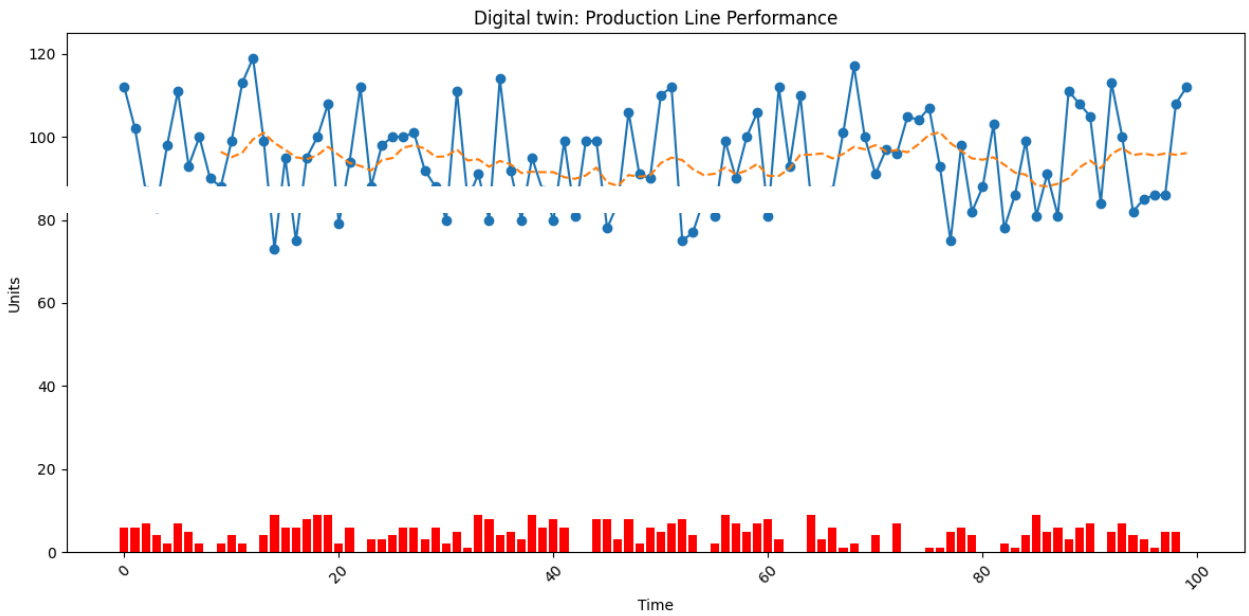
	Time	Units Produced	Defective Units	Yield	Efficency
0	0	88	0	88	100.000000
1	1	85	4	81	95.294118
2	2	95	2	93	97.894737
3	3	108	0	108	100.000000
4	4	82	3	79	96.341463

```
df['Predictive Yield'] = df['Yield'].rolling(window=10).mean()
df.head(20)
```

	Time	Units Produced	Defective Units	Yield	Efficency	Predictive Yield
0	0	88	0	88	100.000000	NaN
1	1	85	4	81	95.294118	NaN
2	2	95	2	93	97.894737	NaN
3	3	108	0	108	100.000000	NaN
4	4	82	3	79	96.341463	NaN
5	5	99	4	95	95.959596	NaN
6	6	115	6	109	94.782609	NaN
7	7	98	0	98	100.000000	NaN
8	8	105	2	103	98.095238	NaN
9	9	82	1	81	98.780488	93.5
10	10	98	8	90	91.836735	93.7
11	11	99	9	90	90.909091	94.6
12	12	111	5	106	95.495495	95.9
13	13	86	9	77	89.534884	92.8
14	14	112	2	110	98.214286	95.9
15	15	119	7	112	94.117647	97.6
16	16	118	7	111	94.067797	97.8
17	17	97	1	96	98.969072	97.6
18	18	119	5	114	95.798319	98.7

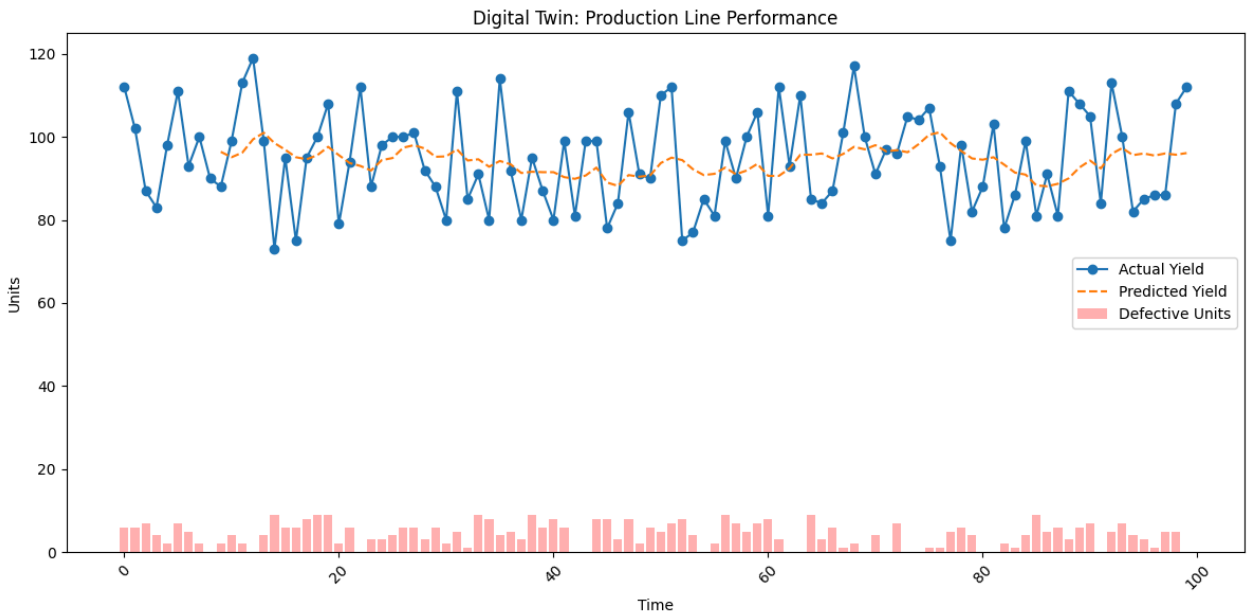
```
plt.figure(figsize=(12,6))
# Plot actual yield
plt.plot(df['Time'], df['Yield'], label='Actual Yield', marker='o
```

```
# Plot predicted yield
plt.plot(df['Time'], df['Predicted_Yield'], label='Predicted Yiel

plt.bar(df['Time'],df['Defective Units'],color='red',label='Defec

plt.title('Digital twin: Production Line Performance')
plt.xlabel('Time')
plt.ylabel('Units')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```




```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Simulate real-time production data
```

```
np.random.seed(42)  # For reproducibility
time_steps = 100
production_data = {
    'Time': np.arange(time_steps),
    'Units_Produced': np.random.randint(80, 120, size=time_steps),
    'Defective_Units': np.random.randint(0, 10, size=time_steps)
}

# Create a DataFrame
df = pd.DataFrame(production_data)

# Calculate yield and efficiency
df['Yield'] = df['Units_Produced'] - df['Defective_Units']
df['Efficiency'] = (df['Yield'] / df['Units_Produced']) * 100

# Digital Twin: Predict future performance using a simple moving a
window_size = 10
df['Predicted_Yield'] = df['Yield'].rolling(window=window_size).me

# Plot the data
plt.figure(figsize=(12, 6))

# Plot actual yield
plt.plot(df['Time'], df['Yield'], label='Actual Yield', marker='o

# Plot predicted yield
plt.plot(df['Time'], df['Predicted_Yield'], label='Predicted Yield

# Highlight defective units
plt.bar(df['Time'], df['Defective_Units'], color='red', alpha=0.3,

# Add labels and legend
plt.title('Digital Twin: Production Line Performance')
plt.xlabel('Time')
plt.ylabel('Units')
plt.legend()
plt.xticks(rotation=45)
plt.tight_layout()

# Show the plot
plt.show()
```




```
#### Data Analysis


import pandas as pd

# Load production data
data = {
    'Machine': ['A', 'B', 'C', 'A', 'B', 'C'],
    'Output': [100, 150, 200, 110, 160, 210],
    'Defects': [5, 3, 2, 4, 2, 1]
}

df = pd.DataFrame(data)
display(df)
```

	Machine	Output	Defects
0	A	100	5
1	B	150	3
2	C	200	2
3	A	110	4
4	B	160	2
5	C	210	1

```
# Calculate total output and defects per machine
summary = df.groupby('Machine').sum()
summary['Defect Rate'] = (summary['Defects'] / summary['Output'])

print(summary)
```

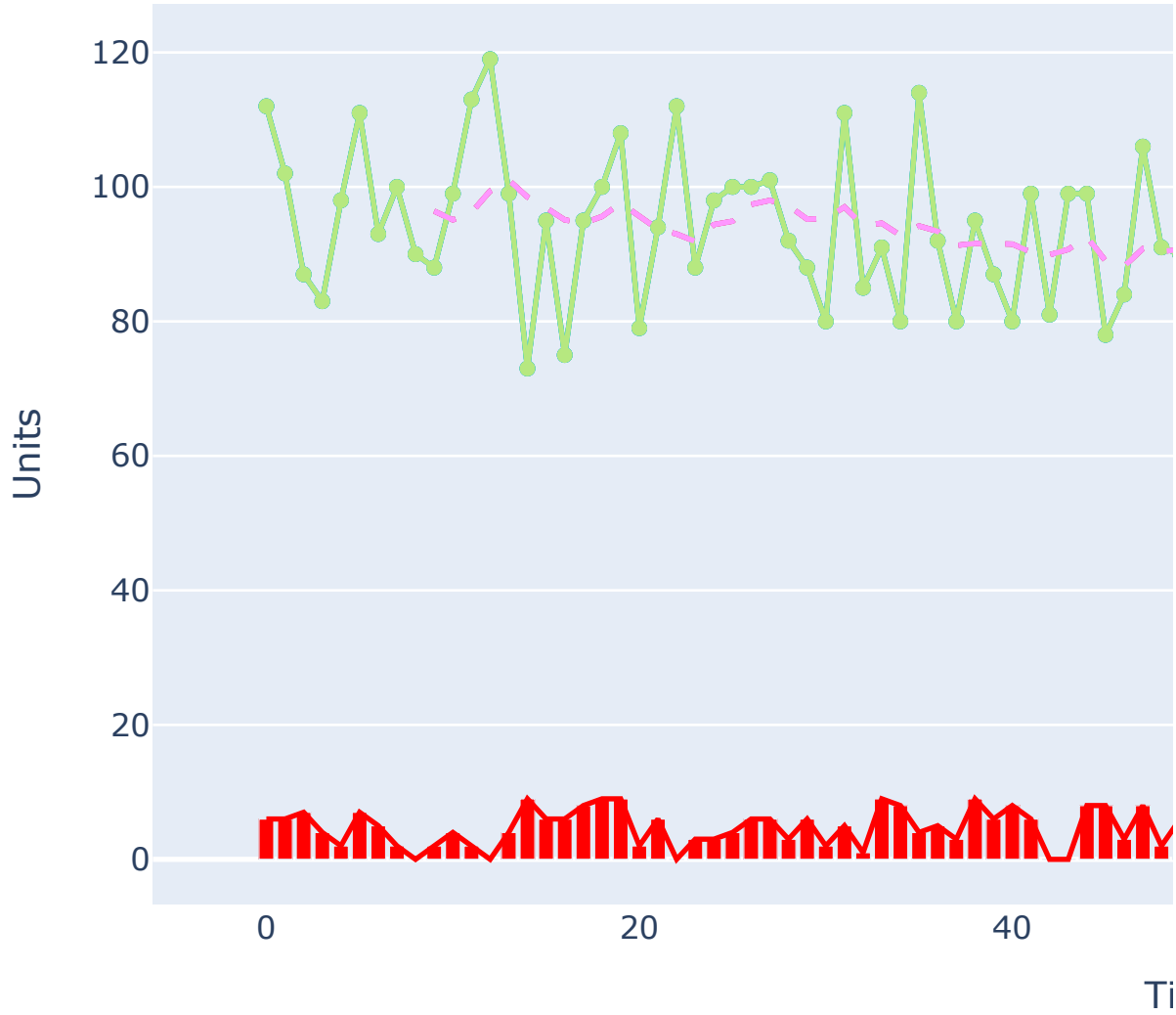
```
          Output  Defects  Defect Rate
Machine
A            210        9     4.285714
B            310        5     1.612903
C            410        3     0.731707
```

```
summary.describe(percentiles=[0.8,0.85,0.89,0.90,0.95])
```

	Output	Defects	Defect Rate
count	3.0	3.000000	3.000000
mean	310.0	5.666667	2.210108
std	100.0	3.055050	1.850738
min	210.0	3.000000	0.731707
50%	310.0	5.000000	1.612903
80%	370.0	7.400000	3.216590
85%	380.0	7.800000	3.483871
89%	388.0	8.120000	3.697696
90%	390.0	8.200000	3.751152
95%	400.0	8.600000	4.018433
max	410.0	9.000000	4.285714

```
### Interactive graph


import plotly.express as px


import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Simulate real-time production data
np.random.seed(42)  # For reproducibility
time_steps = 100
production_data = {
    'Time': np.arange(time_steps),
    'Units_Produced': np.random.randint(80, 120, size=time_steps),
    'Defective_Units': np.random.randint(0, 10, size=time_steps)
}

# Create a DataFrame
```

```
df = pd.DataFrame(production_data)

# Calculate yield and efficiency
df['Yield'] = df['Units_Produced'] - df['Defective_Units']
df['Efficiency'] = (df['Yield'] / df['Units_Produced']) * 100

# Digital Twin: Predict future performance using a simple moving a
window_size = 10
df['Predicted_Yield'] = df['Yield'].rolling(window=window_size).me


import plotly.express as px
import plotly.graph_objects as go


fig = go.Figure()


fig.add_trace(go.Scatter(x = df['Time'],y=df['Yield'],mode = 'line
fig.add_trace(go.Scatter(x = df['Time'],y=df['Predicted_Yield'],mo

fig.add_trace(go.Bar(x = df['Time'],y=df['Defective_Units'],name=

fig.update_layout(
    title = 'Digital twin: Production line performance',
    xaxis_title = 'Time',
    yaxis_title = 'Units'
)

fig.show()
```




```
# ### Libraries Name/List based on type of usage


# 1. Visualization Task

# A. matplotlib
# B. plotly
# C. Searborn
# D. Bokeh
# E. Altair
```

```
# 2. Data Manipulation and Analysis
# A. pandas
# B. numpy
# C. Dask

# 3. Statistical Analysis and simulation
# A. SciPy
# B. Statsmodels

# 4. Networking and communication

# A. request
# B. socket

# 5. Computer Vision
# A. CV2


### Read data from CSV


df = pd.DataFrame(production_data)
display(df)
```

	Time	Units_Produced	Defective_Units
0	0	118	6
1	1	108	6

Start coding or generate with AI.

```
df.to_csv("production_data.csv",index=False)
```

4	4	100	2

```
prod_data = pd.read_csv("production_data.csv")
```

```
prod_data.shape
```

```
(100, 3)
```

97	97	91	5